

รายงานความคืบหน้า

1 หัวข้อและปัญหา

1.1 หัวข้อ

Raycaster Implementation ตาม Wolfenstein 3D ด้วยภาษา C และ OpenGL



รูปที่ 1 เกม Wolfenstein 3D

1.2 ปัญหา

หากนึกถึงการ render 3 มิติ วิธีแรกที่น่าจะขึ้นมาในหัวคือให้แต่ละ pixel บนหน้าจอยิงรังสี (Ray) ออกไปในทิศทางของ pixel นั้น โดยใช้ $\cos$, $\sin$ ในการเก็บทิศทางของรังสี แล้วเช็คว่ารังสีไปกระทบสิ่งใด (ในที่นี้คือ object) วิธีเช็คที่ตรงไปตรงมาที่สุดคือให้รังสีเดินไปที่ละก้าวเล็กๆ (step) แล้วถามทุกก้าวว่าถึง object หรือยัง ก้าวแรกที่ชน object คือสิ่งที่ pixel นั้นมองเห็น วิธีนี้ตรงไปตรงมาและให้ภาพที่ถูกต้อง แต่เมื่อนำมาทำจริงบนเครื่องปี 1992 การเดินรังสีทีละก้าวจะทำให้เกิดปัญหาด้านการคำนวณ

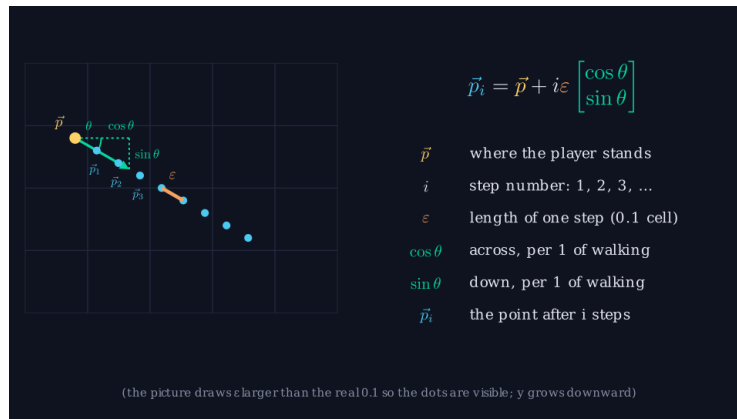
1.2.1 Ray walk using Steps

Ray walk using Steps คือวิธีหาว่า Ray ชน object ที่ไหน โดยให้ Ray ขยับไปข้างหน้าทีละ ε ช่อง (Step) แล้วเช็คทุก Step ว่าชน object หรือยัง ในวิธีเริ่มต้นนี้เราจะมี Ray ออกจากทุก Pixel ของภาพ โดยแต่ละเส้น Ray เริ่มที่ตำแหน่งของผู้เล่น $\vec{p}$ และทำมุม θ กับแกน x เมื่อ Ray ชน object ก็จะได้สีของ Pixel นั้นมา สมการการเดินของ Ray หลังจากผ่านไป i ก้าวเป็นดังนี้

$$\vec{p}_i = \vec{p} + i\varepsilon \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} \quad (1)$$

ความหมายตัวแปร

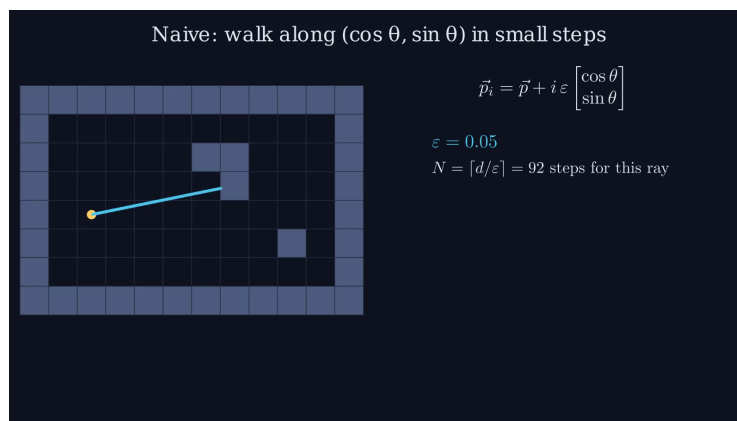
1. $\vec{p}_i$: ตำแหน่งของ Ray ณ Step i
2. $\vec{p}$: ตำแหน่งจุดเริ่มต้นของ Ray (ตำแหน่งผู้เล่น/กล้อง)
3. i : จำนวนของ Step
4. ε : ขนาดของ Step
5. θ : มุมที่ Ray ทำกับแกน x
6. $(\cos \theta, \sin \theta)$: Vector 1 หน่วยที่เป็นทิศทางของ Ray นั้นๆ



รูปที่ 2 การเดินของเส้น Ray

1.2.2 Calculation per image

ปัญหาของวิธีนี้คือ Operation ที่ต้องทำให้ภาพเพียง 1 Frame



รูปที่ 3 ตัวอย่างแผนที่

สมมติให้มีแผนที่ดังรูป ให้จุดสีเหลืองคือผู้เล่น มีขนาดของ Step คือ 0.05 (เฉลี่ย Step ก่อนที่จะชน object ของ 1 Ray คือ 92 Step) และขนาดภาพคือ 320×200 Pixel จะต้องใช้ในการคำนวณมากกว่า

$$(320 \times 200) \text{ Rays} \times 92 \text{ Steps} = 5,888,000 \text{ Ray calculation} \quad (2)$$

ซึ่งยังไม่รวมการบวกและคูณแบบ Floating Point, การหา cos และ sin ของแต่ละ Ray

1.2.3 Undetermined step size

การเลือกขนาดของ Step ที่เหมาะสมเป็นไปได้ยากเพราะหากมากไป Ray ก็ จะทะลุ object ถ้าน้อยไปจำนวน Calculation ก็จะมีมากขึ้น

1.2.4 What games need

ถ้าอ้างอิง Processor ในสมัยปี 1992 อย่าง i386 ที่เป็น Model 33MHz จะใช้เวลา ประมาณ 6 วินาทีในการคำนวณภาพ 1 Frame ตาม Environment ใน 1.2.2 แต่ในการที่ เราจะหลอกตาคนเล่นได้ว่าภาพไม่ใช่ภาพนิ่งๆจะต้องใช้ภาพประมาณ 24 FPS (Frame per second) ทำให้การคำนวณวิธีนี้เป็นไปไม่ได้เลยในการนำมาทำเกม

2 แนวทางแก้ปัญหา

2.1 จำกัดแผนที่เป็น 2D

กำหนดแผนที่ให้เป็นได้แค่ตาราง 2 มิติ ที่เป็นได้แค่กำแพงหรืออากาศ กำแพงในทุกช่องจะมีความสูงเท่ากัน และผู้เล่นจะไม่สามารถหันหน้าขึ้นลงได้ ทำให้ตัดเรื่องการยิง Ray ไปชนกำแพงที่มีความสูงหลายชั้นออกไปได้ สำหรับพื้นและเพดานจะให้ล๊อคเป็นสี่เหลี่ยมไม่ต้องไปคำนวณเพิ่มเติม

2.2 ยิง Ray แคในแนว Column ของภาพ

หลังจากที่เราล๊อคเรื่องความสูงของกำแพงไปแล้ว จะสังเกตได้ว่า Ray ทุกเส้นที่อยู่ใน Column เดียวกัน จะไปชนที่กำแพงอันเดียวกันเสมอ ทำให้เราสามารถเปลี่ยนจากยิง Ray ออกจากทุก Pixel เป็นทุก Column แทนได้

2.2.1 Vector ของตัวผู้เล่นและกล้อง

ผู้เล่น/กล้องยังคงใช้ Vector $\vec{p}$ เป็นการเก็บตำแหน่งของตัวเองเหมือนเดิม และจะมีการเก็บ Vector $\vec{d}$ เพิ่มซึ่งคือทิศทางการหันหน้าของตัวผู้เล่นเป็น Vector ยาว 1 หน่วย

ในการสร้างระนาบกล้องจะมีการใช้ตัวแปรเพิ่มคือ Rotation Matrix J และค่าคงตัว 1 ค่า (ให้เป็น β)

ค่าคงตัว β จะนำมาใช้กำหนดค่า FOV ของกล้องผ่านสมการ

$$\text{FOV} = 2 \arctan(\beta) \quad (3)$$

ถ้าใช้ค่าเดียวกับเกม Wolfenstein 3D ($\beta = 0.66$) จะได้ $\text{FOV} = 2 \arctan(0.66) \approx 66.8^\circ$

Vector ระนาบกล้อง $\vec{P}$ จะสร้างจากการนำค่าทั้ง 3 ค่านั้นมาคูณกันเพื่อสร้าง Vector ใหม่ 1 ตัว

$$\vec{P} = \beta J \vec{d}, \quad J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \quad (4)$$

ความหมายตัวแปร

1. $\vec{P}$: Vector ระนาบกล้อง ชี้ไปทางขอบขวาของจอ
2. β : ค่าคงตัวที่เกิดจากการคิด FOV
3. J : Rotation Matrix สำหรับส่งให้ $\vec{P}$ ตั้งฉากกับ $\vec{d}$
4. $\vec{d}$: Vector ทิศทางการหันของผู้เล่น

2.2.2 Vector ทิศทางของ Ray

เราสามารถหาพิกัดของ Ray ของแต่ละ Column ได้ผ่านสมการ

$$c = \frac{2x}{W} - 1 \quad (5)$$

ความหมายตัวแปร

1. c : พิกัดของ Ray มีค่าอยู่ระหว่าง -1 (ขอบซ้ายของจอ) และ 1 (ขอบขวาของจอ)
2. x : Column ของ Ray ปัจจุบัน
3. W : ขนาดความกว้างของจอ

แล้วเราก็สามารถนำ c มาหาต่อเป็นทิศทาง Ray ยิ่งออกไปได้ผ่านสมการ

$$\vec{r} = \vec{d} + c \vec{P} \quad (6)$$

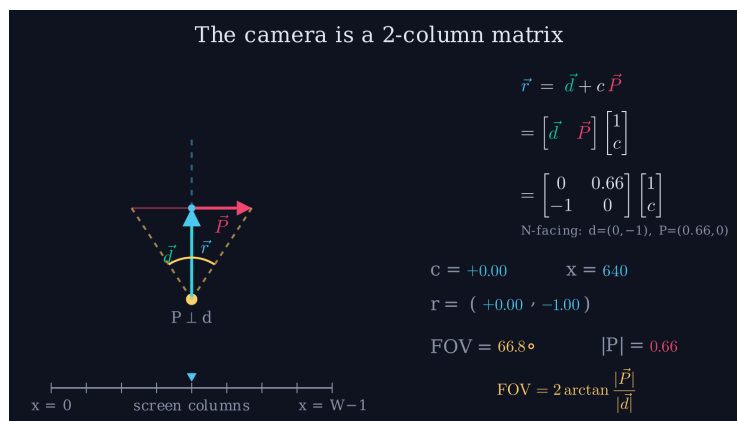
ความหมายตัวแปร

1. $\vec{r}$: Vector ทิศทางของ Ray นั้นๆ
2. $\vec{d}$: Vector ทิศทางการหันของผู้เล่น
3. c : พิกัดของ Ray เมื่อเอา Column มาเทียบกับพิกัดบนจอ
4. $\vec{P}$: Vector ระบายกล้อง

แต่เราต้องเปลี่ยนการเก็บ Vector $\vec{d}$ และ $\vec{P}$ ให้มาอยู่ในรูปแบบของ Matrix 2×2 M ทำหน้าที่เป็น Matrix กล้องแทนที่จะเป็น Vector ที่อยู่แยกกัน ทำให้สมการของ Vector $\vec{r}$ มาอยู่ในรูปของ

$$\vec{r} = \vec{d} + c \vec{P} = \underbrace{\begin{bmatrix} \vec{d} & \vec{P} \end{bmatrix}}_M \begin{bmatrix} 1 \\ c \end{bmatrix} \quad (7)$$

M จึงเป็น transformation matrix ที่แปลงตำแหน่งบนจอ c ให้เป็นทิศทางของ Ray บนแผนที่ด้วยการคูณเพียงครั้งเดียว



รูปที่ 4 Matrix กล้อง (M) และ Vector $\vec{r}$ ของ Ray

2.2.3 การหมุนกล้อง

เมื่อเราแปลงรูป Vector มาให้อยู่ในรูปของ Matrix M จะทำให้เราสามารถหมุนกล้องได้ด้วยผ่านการคูณ Rotation Matrix (R)

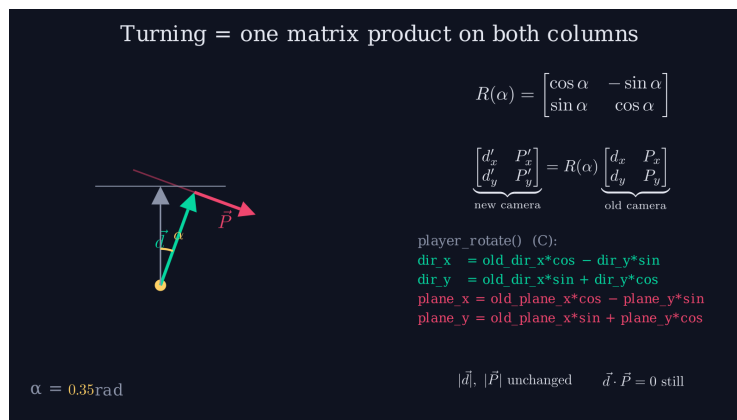
$$M' = R(\alpha) M, \quad R(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix} \quad (8)$$

ความหมายตัวแปร

1. M : Matrix กล้องก่อนหมุน
2. M' : Matrix กล้องหลังหมุน
3. R : Rotation Matrix สำหรับหมุน Matrix กล้อง
4. α : มุมที่จะหมุนกล้อง (ใช้เป็น Radian)

วิธีนี้จะดีกว่าวิธีแรกสุดด้วยเพราะหา $\sin$ และ $\cos$ ให้ Matrix กล้องแค่รอบเดียว แทนที่จะต้องมาหาให้กับ Ray ทุกเส้น

R เป็น orthogonal matrix: $R^T R = I$ (I คือ identity matrix) และ $\det R = 1$ จึงหมุนโดยไม่ยืดและไม่สะท้อน หลังหมุนก็ครั้งก็ยังได้ $|\vec{d}| = 1$, $|\vec{P}| = \beta$ และ $\vec{d} \cdot \vec{P} = 0$



รูปที่ 5 การหมุนกล้อง ผ่านการคูณ Matrix R

2.3 ใช้ DDA แทน Step

วิธี Step จะมีปัญหาเรื่องว่าจะเลือกจำนวนเยอะหรือจะปล่อยให้กำแพงบางมาก ๆ หลุดไป แต่วิธี DDA (Digital Differential Analyzer) ทำงานโดยเดินทีละ 1 ช่อง และจำไว้ว่าแต่ละแกน (x และ y) ใช้ระยะตามเส้น Ray ไปแล้วเท่าไร จากนั้นเทียบสองแกน แกนไหนใช้ไปมากกว่า ก็ให้เดินด้วยอีกแกน แล้วเช็คกว่าช่องที่เข้าไปเป็นกำแพงหรือยัง ทำซ้ำไปเรื่อยๆ จนถึงกำแพง

สมการหาจุดบน Ray คือ

$$\vec{q}(t) = \vec{p} + t \vec{r} \quad (9)$$

ความหมายตัวแปร

1. $\vec{q}$: จุดบนเส้น Ray
2. $\vec{p}$: ตำแหน่งของผู้เล่น (จุดกำเนิด Ray)
3. t : ระยะที่เดินตาม Ray
4. $\vec{r}$: Vector ทิศทางของ Ray เส้นนั้น

การเดิน 1 ช่องมีการเดินสองแบบแยกกัน คือเดินในแกน x และเดินในแกน y เส้นกริดอยู่ที่จำนวนเต็ม จึงต้องรู้ว่าเดิน 1 ช่องในแต่ละแกนใช้ t เพิ่มเท่าไร ซึ่งได้จากสมการเส้นตรง โดยแยกเป็นแต่ละแกน จัดรูปให้เป็น t แล้ว diff เทียบกับพิกัด:

$$q_x = p_x + t r_x \quad \Rightarrow \quad t = \frac{q_x - p_x}{r_x} \quad \Rightarrow \quad \frac{dt}{dq_x} = \frac{1}{r_x} \quad (10)$$

นั่นคือเดิน 1 ช่องในแกน x ต้องเพิ่ม t เท่ากับ $1/|r_x|$ (ใช้ค่าสัมบูรณ์เพื่อให้เป็นบวกเสมอ แม้ Ray จะไปทางซ้ายหรือขึ้น) แกน y ก็ทำนองเดียวกัน:

$$\Delta t_x = \frac{1}{|r_x|}, \quad \Delta t_y = \frac{1}{|r_y|} \quad (11)$$

ความหมายตัวแปร

1. $\Delta t_x, \Delta t_y$: ค่า t ที่ต้องเพิ่มเพื่อให้ Ray เลื่อนไป 1 ช่องในแกน x หรือแกน y (คือระยะห่างของ t ระหว่างการตัดเส้นตั้งสองเส้นติดกัน หรือเส้นนอนสองเส้นติดกัน)
2. r_x, r_y : องค์ประกอบแกน x และแกน y ของ $\vec{r}$
 ดังนั้นแต่ละแกนมีค่า t ที่ Ray ขนเส้นกริดเรียงเป็นลำดับเลขคณิต: $s_x + k \Delta t_x$ และ $s_y + k \Delta t_y$ ($k = 0, 1, 2, \dots$) การเดินเริ่มต้นคือเดินไปถึงเส้นกริดเส้นแรกของแต่ละแกนก่อน ซึ่งได้ค่า s_x, s_y คือส่วนของช่องที่ยังอยู่ข้างหน้าคุณ Δt (m คือช่องที่ผู้เล่นยืนอยู่):

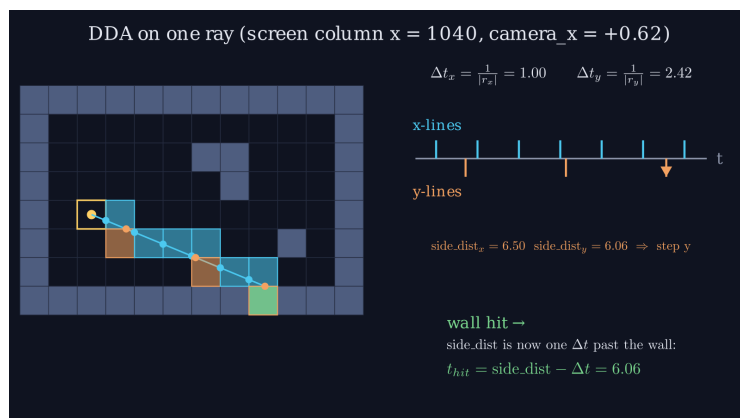
$$s_x = \begin{cases} (p_x - m_x) \Delta t_x & r_x < 0 \\ (m_x + 1 - p_x) \Delta t_x & r_x \geq 0 \end{cases} \quad (12)$$

ความหมายตัวแปร

1. s_x : ค่า t ที่ Ray ขนเส้นตั้งเส้นแรก (s_y เป็นค่าของเส้นนอนในทำนองเดียวกัน)
2. p_x : พิกัด x ของผู้เล่น
3. m_x : เลขช่องแนว x ที่ผู้เล่นยืนอยู่
4. กรณี $r_x < 0$: Ray ไปทางซ้าย เส้นแรกคือขอบซ้ายของช่อง ห่างออกไป $p_x - m_x$
5. กรณี $r_x \geq 0$: Ray ไปทางขวา เส้นแรกคือขอบขวาของช่อง ห่างออกไป $m_x + 1 - p_x$
6. การคูณด้วย Δt_x : แปลงระยะบนแผนที่ (หน่วยช่อง) ให้เป็นค่า t

จากนั้น DDA เดินตามวิธีที่อธิบายไว้ข้างต้น โดยทุกครั้งที่เดิน ให้บวก Δt ของแกนที่เพิ่งเดินเพิ่มในค่า t ของแกนนั้น และเพราะทุกช่องที่ Ray ผ่านถูกเช็คครบ จึงไม่มีกำแพงบางใดหลุดรอดไปได้

ตัวอย่าง ผู้เล่นอยู่ที่ $\vec{p} = \langle 2.5, 4.5 \rangle$ หันไปทางตะวันออก $\vec{d} = \langle 1, 0 \rangle$ จึงได้ $\vec{P} = \langle 0, 0.66 \rangle$ คอลัมน์ $x = 1040$ ของจอกว้าง $W = 1280$ ได้ $c = 0.625$ และ $\vec{r} = \langle 1, 0.4125 \rangle$ จะได้ $\Delta t_x = 1.00$ และ $\Delta t_y = 2.42$ DDA ใช้ 9 รอบก็ถึงกำแพงที่ช่อง (8, 7) ได้ $t = 6.06$ แทนที่จะเดินทีละ 0.05 ซึ่งต้องใช้ราว 130 ก้าว



รูปที่ 6 DDA บน Ray เส้นเดียว: สองลำดับของการตัดเส้นกริด และ $t_{hit} = 6.06$

2.4 ใช้ 3D Projection

เมื่อรู้ว่า Ray ชนกำแพงที่ระยะเท่าไร ก็แปลงเป็นความสูงกำแพงบนจอได้ด้วย projection ของที่อยู่ใกล้จึงดูเล็กลง แต่ความลึกที่ใช้ต้องเป็น perpendicular distance (ไม่ใช่ระยะตาม Ray) เพื่อไม่ให้ผนังเรียบดูโค้งเป็นภาพตาปลา (fisheye) ซึ่งหาได้จาก dot product

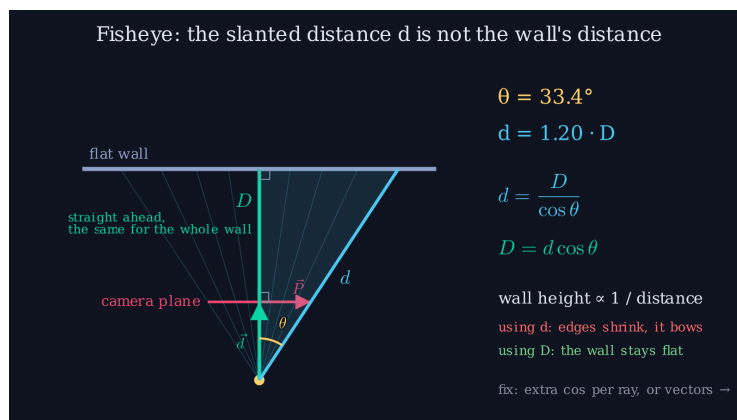
2.4.1 dot product และ perpendicular distance

ความลึกที่ใช้วาดกำแพงต้องไม่ใช่ระยะตาม Ray ตรงๆ ต้องใช้ perpendicular distance D คือระยะตั้งฉากกับระนาบกล้อง (วัดตามทิศ $\vec{d}$) แทน เพราะผนังเรียบที่ขนานกับระนาบกล้องอยู่ห่างจากระนาบเท่ากันทุกคอลัมน์ แต่ระยะตาม Ray ยิ่งใกล้ขอบจอยิ่งยาว ถ้าเอามาใช้ ขอบผนังจะดูเล็กกว่ากลางผนัง ผนังเรียบจึงดูโค้งเหมือนภาพตาปลา (fisheye effect) Ray ที่เบนจากทิศ $\vec{d}$ ไปมุม θ ยาวกว่า D อยู่ $1/\cos\theta$ เท่า (ตัว d ในสูตรนี้และในภาพคือความยาว ไม่ใช่ Vector ทิศทาง $\vec{d}$):

$$d = \frac{D}{\cos\theta} \Leftrightarrow D = d \cos\theta \quad (13)$$

ความหมายตัวแปร

1. D : perpendicular distance จากผู้เล่นถึงกำแพง วัดตามทิศ $\vec{d}$ (ไปข้างหน้า)
2. d : ระยะเฉียงตาม Ray ถึงกำแพง (เป็นความยาว ไม่ใช่ Vector)
3. θ : มุมระหว่าง Ray กับทิศ $\vec{d}$



รูปที่ 7 ผนังเรียบที่ D คงที่ แต่ระยะเฉียง d เพิ่มขึ้นตามมุม θ เส้นแดงคือระนาบกล้อง

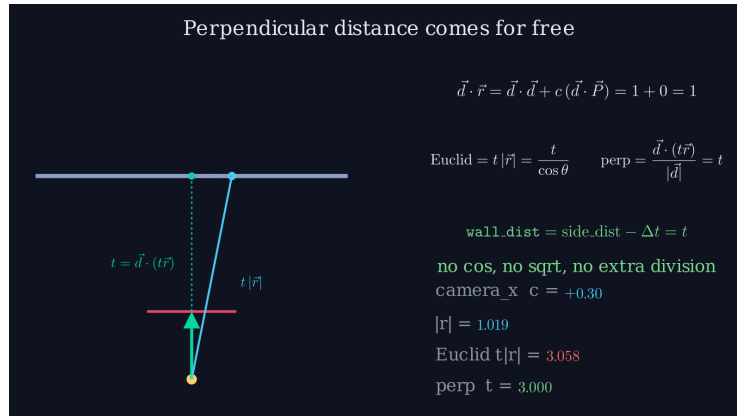
แต่เราไม่ต้องคำนวณ $\cos$ เพราะ Ray $\vec{r} = \vec{d} + c\vec{P}$ มีส่วนไปข้างหน้า (ตาม $\vec{d}$) ยาว 1 เสมอ ส่วน $c\vec{P}$ ตั้งฉากกับ $\vec{d}$ จึงไม่ช่วยให้ไปข้างหน้าเพิ่ม dot product บอกสิ่งนี้ได้ตรงๆ คือส่วนของ Vector ที่ชี้ไปตาม $\vec{d}$ (เพราะ $|\vec{d}| = 1$):

$$\vec{d} \cdot (t\vec{r}) = t(\vec{d} \cdot \vec{d} + c\vec{d} \cdot \vec{P}) = t(1 + c \cdot 0) = t \quad (14)$$

ความหมายตัวแปร

1. $t\vec{r}$: Vector จากผู้เล่นไปถึงจุดที่ Ray ชนกำแพง
2. $\vec{d} \cdot \vec{d} = 1$: เพราะ $\vec{d}$ ยาว 1 หน่วย
3. $\vec{d} \cdot \vec{P} = 0$: เพราะ $\vec{d}$ ตั้งฉากกับ $\vec{P}$

4. ผลลัพธ์ t : ส่วนของ $t\vec{r}$ ที่ชี้ไปข้างหน้าตาม $\vec{d}$ ซึ่งคือ perpendicular distance
 เพราะ $\vec{d} \cdot \vec{d} = 1$ และ $\vec{d} \cdot \vec{P} = 0$ (ตั้งฉากกัน) จึงได้ $\vec{d} \cdot \vec{r} = 1$ สำหรับทุกคอลัมน์
 perpendicular distance จึงเท่ากับ t_{hit} พอดี เรียกค่านี้ว่าความลึก $z = t_{\text{hit}}$ ตรงกับที่ใช้
 กด้วยมุมได้: $|\vec{r}| = 1/\cos\theta$ จึง $t|\vec{r}|\cos\theta = t$



รูปที่ 8 ระยะเฉียง $t|\vec{r}|$ กับระยะตั้งฉาก $t = \vec{d} \cdot (t\vec{r})$

2.4.2 inverse Matrix และความสูงกำแพง

เราใช้ projection แปลงโลก 2D ของแผนที่ให้เป็นภาพ 3D บนจอ โดยแปลงจุดในโลกเป็นพิกัดกล้องด้วย inverse Matrix M^{-1} แล้วหารด้วยความลึก ของที่อยู่ไกลจึงดูเล็กลง:

$$\begin{bmatrix} a \\ b \end{bmatrix} = M^{-1}(\vec{q} - \vec{p}), \quad c = \frac{b}{a}, \quad x = \frac{W}{2}(1 + c) \quad (15)$$

ความหมายตัวแปร

1. $\vec{q}$: จุดในโลก (เช่น มุมของกำแพง) ที่ต้องการรู้ว่าจะปรากฏที่คอลัมน์ไหนของจอ
2. $\vec{p}$: ตำแหน่งผู้เล่น และ $\vec{q} - \vec{p}$ คือ Vector จากผู้เล่นไปยังจุด $\vec{q}$
3. M^{-1} : inverse Matrix ของ M ซึ่งแปลงทิศบนแผนที่กลับเป็นพิกัดกล้อง
4. a, b : พิกัดกล้องของ $\vec{q} - \vec{p}$ โดย a คือไปข้างหน้า และ b คือไปข้าง
5. c : ตำแหน่งของคอลัมน์ในช่วง -1 ถึง $+1$
6. x : คอลัมน์ของจอ (พิกเซล)
7. W : ความกว้างของจอเป็นพิกเซล

การหาร $c = b/a$ ทำให้ของที่อยู่ไกลดูเล็กลง การยิง Ray คือการทำทางตรงข้ามนี้: เริ่มจาก c แล้วสร้าง $\vec{r} = M\langle 1, c \rangle$ จากนั้นค้นหาจุดที่ Ray ชน จุด $\vec{q} = \vec{p} + t\vec{r}$ มี $a = t$ และ $b = tc$ ซึ่งเป็นเหตุผลเดียวกับที่ความลึกเท่ากับ t

แต่ความลึก a ที่ใช้ต้องไม่ใช่ระยะตาม Ray ตรงๆ ต้องเป็น perpendicular distance เพื่อไม่ให้เกิด fisheye effect (หัวข้อ 2.4.1) และ $a = t_{\text{hit}} = z$ ตามที่แสดงไว้แล้ว

ความสูงของกำแพงบนจอเป็นความสัมพันธ์แบบผกผันกับความลึก z คือยิ่งไกลยิ่งเตี้ย กำแพงที่อยู่ห่าง $z = 1$ จะสูงเต็มจอ H พิกเซล และถ้าอยู่ห่างเป็น z เท่า ความสูงจะลดลงเหลือ H/z (เช่น ไกลขึ้น 2 เท่า สูงเหลือครึ่งหนึ่ง) เราได้ความสูงกำแพงในหน่วยพิกเซล

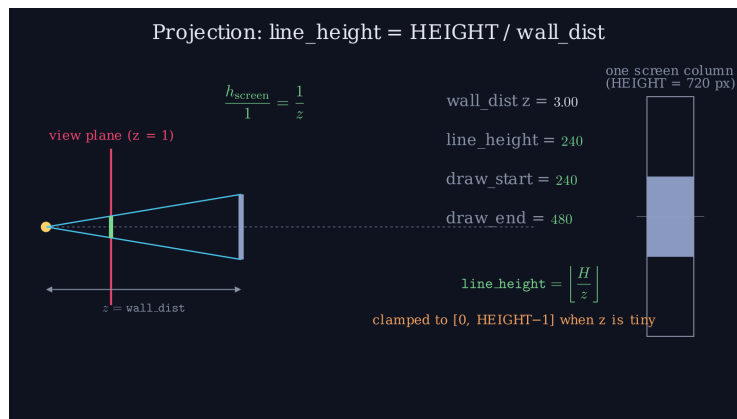
(และจัดให้อยู่กึ่งกลางจอ):

$$lh = \left\lfloor \frac{H}{z} \right\rfloor, \quad \text{draw_start} = \frac{H}{2} - \frac{lh}{2}, \quad \text{draw_end} = \frac{H}{2} + \frac{lh}{2} \quad (16)$$

ความหมายตัวแปร

1. z : ความลึก (perpendicular distance ถึงกำแพง) โดย $z = t_{\text{hit}}$
2. H : ความสูงของจอเป็นพิกเซล (ตัวอย่างใช้ 720)
3. lh : ความสูงของเส้นกำแพงในคอลัมน์นั้น (พิกเซล)
4. draw_start , draw_end : แถวบนสุดและล่างสุดที่ใช้วาดกำแพง นับจากขอบบนของจอ โดยจัดให้อยู่กึ่งกลางที่ $H/2$

ตัวอย่าง $H = 720$: $z = 3$ ได้ 240 แถว Ray ในหัวข้อ 2.3 ($z = 6.06$) ได้ 118 แถว



รูปที่ 9 ความสูงกำแพงบนจอ $lh = \lfloor H/z \rfloor$ ยิ่งไกลยิ่งเตี้ย

2.5 การเคลื่อนที่ของผู้เล่น

ตำแหน่งของผู้เล่นคือ Vector $\vec{p} = \langle x, y \rangle$ ดังนั้นการเดินก็คือการบวกระยะที่เดินไปในแนว x และ y เข้ากับตำแหน่งเดิม

$$\begin{bmatrix} x \\ y \end{bmatrix} \leftarrow \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (17)$$

ความหมายตัวแปร

1. x, y : ตำแหน่งของผู้เล่นบนแผนที่ ซึ่งคือ $\vec{p}$
2. $\Delta x, \Delta y$: ระยะที่เดินไปในแนว x และ y ใน Frame นี้
3. ลูกศร $\leftarrow$ หมายถึงกำหนดค่าใหม่ให้ตัวแปรทางซ้าย

ค่า $\langle \Delta x, \Delta y \rangle$ ได้จากทิศทางที่ผู้เล่นหันอยู่ ปุ่ม W/S เดินไปหน้าหรือถอยหลังตาม $\vec{d}$ และปุ่ม D/A เดินข้างตาม $J\vec{d}$ (คือ $\vec{d}$ ที่หมุน 90°) โดยคูณด้วยระยะที่เดินใน Frame นี้ s

$$\text{W/S: } \vec{p} \leftarrow \vec{p} \pm s \vec{d}, \quad \text{D/A: } \vec{p} \leftarrow \vec{p} \pm s J\vec{d} \quad (18)$$

ความหมายตัวแปร

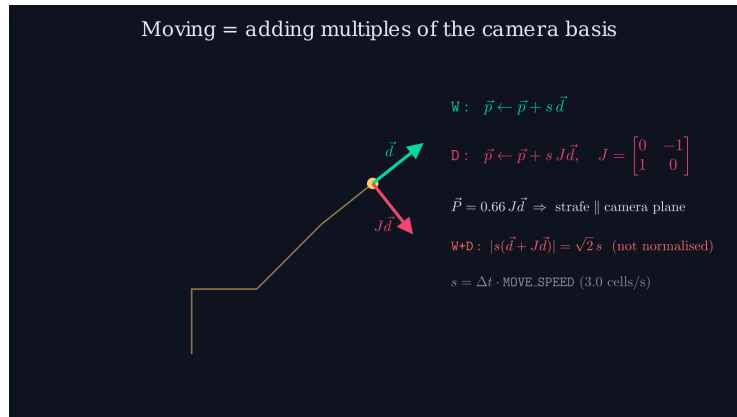
1. s : ระยะที่เดินใน Frame นี้ $= dt \cdot \text{MOVE_SPEED}$ (dt คือเวลาที่ผ่านไปจาก Frame ก่อนการคูณด้วย dt ทำให้เดินเร็วเท่ากันไม่ว่า Frame จะเร็วหรือช้า)
2. $\pm$: เครื่องหมาย + เมื่อกด W หรือ D และเครื่องหมาย - เมื่อกด S หรือ A

3. $\vec{d}$: Vector ทิศทางของผู้เล่น (ไปหน้า)

4. $J\vec{d}$: $\vec{d}$ ที่ถูกหมุน 90° (ไปข้าง)

$\vec{d}$ กับ $J\vec{d}$ ตั้งฉากกันและยาว 1 หน่วยเท่ากัน จึงเป็น orthonormal basis ทำให้การเดินหน้ากับการเดินข้างไม่รบกวนกัน

การหมุนกลิ้ง (หัวข้อ 2.2.3) ก็ใช้ dt เช่นเดียวกัน คือมุม α ที่หมุนในแต่ละ Frame ได้จากการคูณด้วย dt



รูปที่ 10 การเดินคือการบวก $s\vec{d}$ หรือ $sJ\vec{d}$ เข้ากับ $\vec{p}$

การชนกำแพง: ผู้เล่นเป็นสี่เหลี่ยมเล็กๆ โปรแกรมตรวจสอบมุม $(x \pm 0.2, y \pm 0.2)$ และทดลองเดินทีละแกน จึงเผลอตามกำแพงได้ไม่ติด

2.6 หมายเหตุเรื่องการพัฒนา

โปรเจกต์นี้พัฒนาด้วยภาษา C และ OpenGL เพื่อให้สภาพแวดล้อมใกล้เคียงกับยุคของ Wolfenstein 3D (ปี 1992) ไม่พึ่ง framework สำเร็จรูป คณิตศาสตร์ทั้งหมดในรายงานนี้จึงต้องเขียนเองทุกขั้นตอน

3 ทฤษฎีที่ใช้

3.1 ทฤษฎี vector

- vector, linear combination และ basis vector (หัวข้อ 2.2.1, 2.2.2)
ใช้สร้าง Ray ของแต่ละคอลัมน์จาก $\vec{d}$ และ $\vec{P}$ โดยไม่ต้องใช้มุม
- สมการเส้นตรง $\vec{q}(t) = \vec{p} + t\vec{r}$ และ differentiation (หัวข้อ 2.3)
ใช้หาว่าเดิน 1 ช่องใช้ t เพิ่มเท่าไร ($dt/dq_x = 1/r_x$) เพื่อให้ DDA กระโดดตามเส้นกริด
- dot product และ perpendicular distance (หัวข้อ 2.4.1)
ใช้หาความลึกของกำแพงโดยไม่ต้องคำนวณ cos และไม่ให้เกิด fisheye effect
- orthonormal basis (หัวข้อ 2.5)
ใช้ $\vec{d}$ และ $J\vec{d}$ เป็นแกนเดินหน้า/ถอยหลังและเดินข้างของผู้เล่น

3.2 ทฤษฎี matrix

- matrix และ transformation matrix (หัวข้อ 2.2.2)
camera matrix M แปลงตำแหน่งบนจอ c เป็นทิศทางของ Ray บนแผนที่ด้วยการคูณครั้งเดียว
- rotation matrix และ orthogonal matrix (หัวข้อ 2.2.3)
ใช้หมุนกล้องโดยไม่ยืดหรือบิดเบี้ยว คำนวณ sin และ cos แค่ครั้งเดียวต่อการหมุน
- identity matrix (หัวข้อ 2.2.3)
เงื่อนไข $R^T R = I$ ยืนยันว่าการหมุนคงความยาวของ $\vec{d}$ และ $\vec{P}$ และคงมุมระหว่างกัน
- inverse matrix (หัวข้อ 2.4.2)
 M^{-1} แปลงจุดในโลกกลับเป็นพิกัดกล้องเพื่อทำ projection ให้เป็นภาพ 3D